

DAY 1 · 월요일

LLM 의 원리와 이해

AI 에이전트의 핵심인 LLM 의 작동 원리 · 프롬프트 · 한계를 이해한다

담당 세션 (1)

오후 특강 2

14:30-16:00 · 90 분

LLM 의 원리와 이해

전체 과정 로드맵

DAY 1

월요일

LLM 원리와 이해

● 진행 중

DAY 2

화요일

에이전트 원리 + 클로드
코드

DAY 3

수요일

미니 프로젝트 + 리서치
이론 · 팀

DAY 4

목요일

팀 리서치 — 기획 · 구현 ·
실행

DAY 5

금요일

실행 · 분석 + 보고서 ·
발표

진행 단계 : 이론 → 도구 → 실전 → 결과 → 발표

오후 특강 2 14:30-16:00 · 90 분

LLM 의 원리와 이해

LLM 이 무엇을 어떻게 하는지 — 토큰 예측 · 학습 · 프롬프트 · 한계를 이해한다 .

LLM 의 원리와 이해

이 세션에서 다룰 내용

1 언어 모델 간단 역사

2 LLM 이란 + 토큰 · 예측

3 학습 : 사전학습 · 정렬

4 추론 (사고) 모델

5 임베딩 & 의미 검색

6 RAG — 검색 증강 생성

7 프롬프트 엔지니어링

8 생성 제어 · 컨텍스트 윈도우

9 능력과 한계 · 환각

10 모델 선택

11 LLM → 에이전트

이 세션에서 다룰 것

- LLM 의 작동 원리

토큰 예측 · 학습 · 추론

- 외부 지식과 결합

임베딩 · 의미 검색 · RAG → 리서치 에이전트의 토대

- 제어와 한계

프롬프트 · 모델 선택 · 환각 등 주의점

언어 모델 간단 역사



NOTE 초기의 통계 기반 모델에서 신경망을 거쳐 Transformer 로 도약한 흐름이며 , 이어지는 슬라이드에서 각 단계를 차례로 살펴본다 .

word2vec (2013) — 단어를 벡터로

- **무엇**

단어를 의미가 담긴 숫자 벡터로 표현 (Google)

- **왜 중요**

의미 연산이 가능 — 'king - man + woman $\approx$ queen'

- **한계**

단어 단위 · 문맥 (앞뒤 의미) 을 잘 못 담음

NOTE 오늘 학습할 '임베딩' 개념이 바로 이 시기에서 출발한다.

Transformer (2017) — Attention

- **무엇**

논문 'Attention is All You Need' 의 새 신경망 구조

- **핵심**

문장 속 단어 간 관계를 한 번에 · 병렬로 파악 (attention)

- **의의**

이전 방식 (RNN) 보다 빠르고 긴 문맥 처리 → 현대 LLM 의 토대

NOTE 현재의 GPT 와 Claude 를 비롯한 주요 모델은 모두 이 Transformer 구조 위에 만들어져 있다 .

GPT·BERT (2018~) — 사전학습 시대

- GPT

생성형 — 다음 토큰 예측 (말 잇기)

- BERT

이해형 — 빈칸 채우기 (문맥 파악)

- 패러다임

대규모 사전학습 → 작업별 미세조정 · 모델 · 데이터 키우기 시작

NOTE 오늘의 핵심 개념인 '사전학습' 이 이 시기에 자리 잡았다.

GPT-3 → ChatGPT (2020·2022) — 규모와 정렬

- GPT-3 (2020)

175B 파라미터 · few-shot — '규모의 힘' 입증

- ChatGPT (2022.11)

정렬 (RLHF) + 대화 인터페이스

- 의의

전문가 도구 → 누구나 쓰는 AI로 폭발적 대중화

NOTE 방대한 능력을 갖춘 사전학습과 지시를 따르게 하는 정렬이 결합되어 대중화가 이루어진 시점이다.

추론 · 멀티모달 (2024~) — 최신 흐름

- **추론 모델**

바로 답하지 않고 단계적으로 '생각' (o1·DeepSeek-R1)

- **멀티모달**

텍스트 + 이미지 · 음성까지 함께 처리

- **에이전트로**

도구 · 반복을 붙여 '행동하는 AI' → Day 2 주제

NOTE 오늘날 우리가 사용하는 모델들이 바로 이 단계에 해당한다.

LLM이란 무엇인가

- Large Language Model

방대한 텍스트로 학습한 대규모 신경망 — 언어를 생성 · 처리

- 핵심 동작

지금까지의 텍스트로 '다음에 올 토큰'을 확률로 예측

- '이해'가 아니라 '패턴'

의미를 아는 게 아니라 통계적 패턴을 재현하는 것

- 예시

'커피를 마시면 잠이 ____' → 다음에 올 토큰을 확률로 고른다

NOTE LLM은 의미를 이해하는 것이 아니라 통계적으로 '다음 토큰'을 예측한다는 점을 기억해야 한다.

입력에서 출력까지



NOTE 한 번에 한 토큰씩 예측하여 반복적으로 생성하는 방식 (autoregressive) 이며 , 이어서 각 단계를 차례로 살펴본다 .

토큰화 — 텍스트를 토큰으로

- 토큰이란

단어 또는 단어 조각 (글자보다 크고 단어보다 작을 수 있음)

- 왜 토큰

어휘를 효율적으로 다루기 위해 — 모든 단어를 외우지 않아도 됨

- 길이 · 비용

영어 ~4 글자 $\approx$ 1 토큰 · 한글은 더 짧게 $\rightarrow$ 토큰 수가 비용 · 길이를 결정

- 예시

'AI 에이전트' $\rightarrow$ 여러 토큰으로 쪼개짐

- 직접 해보기

platform.openai.com/tokenizer — 문장을 넣어 토큰이 어떻게 쪼개지는지 확인

NOTE 토큰 수가 곧 입력의 길이와 비용을 결정하며, 한글은 영어보다 더 많은 토큰을 사용한다.

예측 — 다음 토큰의 확률

- 무엇을 계산

지금까지의 토큰을 보고 '다음에 올 토큰'의 확률 분포

- 예

'커피를 마시면 잠이 ____' → 안 (0.6) · 온다 (0.2) · ...

- 핵심

여기엔 '정답'이 아니라 '확률'만 있다

NOTE 모델이 다루는 것은 사실이 아니라 '그럴듯함'이며, 이것이 환각이 발생하는 근본 원인이다.

샘플링 — 토큰 하나 선택

- 선택 방식

확률 분포에서 토큰 1 개를 고름

- 결정적 vs 다양

가장 높은 것만 ? 또는 확률대로 무작위 ?

- temperature

이 다양성을 조절하는 손잡이 → 뒤 ' 생성 제어 ' 에서

NOTE 동일한 질문에도 매번 다른 답이 나오는 이유가 바로 이 확률적 선택에 있다 .

반복 — 끝까지 생성

- **이어가기**

고른 토큰을 문장에 붙이고 → 다시 예측 (1~3 반복)

- **autoregressive**

한 번에 한 토큰씩, 끝 신호까지

- **함의**

긴 답일수록 앞에서의 작은 흔들림이 누적될 수 있음

NOTE 따라서 출력이 길어질수록 중간 점검과 검증이 더욱 중요해진다.

사전학습 (Pre-training)

- **방대한 텍스트로 자기지도 학습**

웹 · 책 · 코드에서 ' 다음 토큰 맞추기 ' 를 반복

- **무엇을 배우나**

문법 · 세상 지식 · 추론 패턴 · 문체까지 광범위하게

- **한계의 씨앗**

학습 시점 이후 정보는 모름 · 데이터 편향을 흡수

- **비유**

거대한 빈칸 채우기를 수십억 번 — 그 과정에서 지식이 ' 딸려 ' 학습

NOTE 학습이 이루어진 시점과 데이터의 편향이 모델의 한계로 그대로 남는다 .

정렬 (Alignment)

- 사전학습만으론 부족

'유용하고 · 안전하게 · 지시를 따르도록' 추가로 다듬음

- instruction tuning

지시-응답 예시로 '시키는 대로' 하도록 미세조정

- 인간 피드백 (RLHF 등)

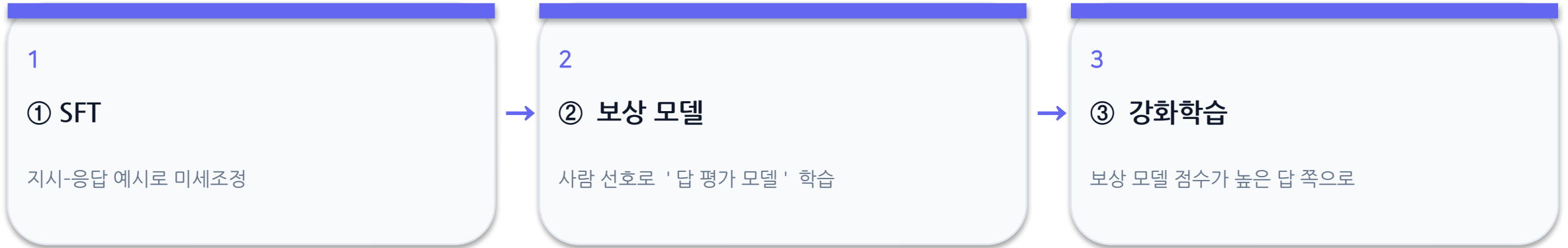
사람이 선호하는 답을 강화 — 도움되고 해롭지 않게

- 사례

InstructGPT(2022) — 정렬한 1.3B 답을 175B GPT-3 보다 사람이 선호 (~71%)

NOTE 기본 능력이 같더라도 정렬 과정이 '지시를 따르는 능력'을 만들어 낸다.

정렬은 어떻게 하나 (RLHF)



NOTE 이어지는 슬라이드에서 정렬의 세 단계를 차례로 살펴본다.

SFT — 지시 따라 하기

- **무엇**

'지시 → 좋은 응답' 예시를 모아 모델을 미세조정

- **효과**

질문에 '말 잇기'가 아니라 '대답'을 하게 됨

- **비유**

모범답안을 따라 쓰며 배우는 단계

보상 모델 — 답을 평가하는 모델

- **무엇**

한 질문에 여러 답을 만들고 , 사람이 좋은 순서로 순위를 매김

- **학습**

그 선호를 학습해 답에 점수를 주는 ' 보상 모델 ' 을 따로 만든다

- **왜**

사람이 매번 평가할 수 없으니 , 답을 자동으로 평가하는 모델이 필요

강화학습 — 점수 높은 쪽으로

- **무엇**

보상 모델 점수가 높아지도록 본 모델을 조금씩 조정

- **결과**

유용 · 안전 · 지시 이행이 강화됨

- **주의**

과하면 아부하거나 뻔한 답만 — 균형 필요

사전학습 vs 정렬 — 무엇이 얼마나 드나

사전학습 (지능)

- 데이터 : 약 15.6 조 토큰 (Llama 3.1, 공개)
- 컴퓨팅 : H100 16,000 여 개 · $3.8e25$ FLOPs (공개)
- 비용 : 약 \$78M~191M, 2023 최전선 (추정)
- GPT-4: '1 억 달러 이상' (Altman, 2023)

정렬 (행동)

- 데이터 : 시연 약 1.3 만 건 (InstructGPT, 공개)
- 인력 : 훈련된 계약직 약 40 명 (InstructGPT)
- 컴퓨팅 : 사전학습의 약 1~2 %
- Llama 2: 사람 선호 비교 약 140 만 건

NOTE 지능은 막대한 컴퓨팅 · 데이터 (수천만 ~ 수억 달러) 로 얻는 반면 , 바람직한 행동은 비교적 적은 양의 정교한 사람 판단으로 형성된다 . (GPT-4·Gemini 수치는 제 3 자 추정치)

추론 (사고) 하는 모델

- 일반 생성 vs 추론

바로 답하지 않고 , 단계적으로 ' 생각 ' 한 뒤 답한다

- 언제 강한가

복잡한 문제 · 다단계 추론 · 코딩

- 트레이드오프

더 정확하지만 더 느리고 비싸다

- 사례

DeepSeek-R1(2025· 오픈)·OpenAI o1→o3 — AIME 78%→96.7%

NOTE 벤치마크 수치는 빠르게 갱신되므로 강의 시점을 기준으로 다시 확인하는 것이 좋다 .

임베딩 & 의미 검색

- 임베딩이란

텍스트를 의미를 담은 숫자 벡터로 변환

- 의미 검색

글자가 달라도 '뜻이 비슷한' 문서를 찾음 (키워드 매칭의 한계 극복)

- 어디에 쓰나

방대한 문헌에서 관련 자료 검색 — 리서치 에이전트의 토대

- 사례

Etsy·Spotify 검색 / 추천이 임베딩 기반

NOTE 임베딩은 키워드의 일치가 아니라 '의미'의 유사성으로 자료를 찾는다.

검색 증강 생성 (RAG)



NOTE 외부 지식을 결합해 환각을 줄이고 최신성을 높이는 방식이며, 이어서 각 단계를 차례로 살펴본다.

질문 — 무엇을 묻나

- **입력**

사용자의 질문이 출발점

- **필요시 분해**

큰 질문이면 검색 가능한 작은 질의로

- **목표**

'무슨 근거를 찾아야 하나'를 정한다

검색 — 관련 문서 찾기

- **방법**

질문을 임베딩해 '의미가 가까운' 문서를 검색

- **대상**

문서 폴더·DB·웹 등 (이번 프로젝트는 로컬 corpus)

- **결과**

관련 문단 몇 개 + 출처

NOTE 오늘 학습한 임베딩이 바로 이 검색 단계에서 활용된다 .

주입 — 근거를 프롬프트에

- **무엇**

찾은 문단을 프롬프트 안에 같이 넣어줌

- **효과**

모델이 ' 기억 ' 이 아니라 ' 눈앞의 근거 ' 로 답하게

- **주의**

너무 많이 넣으면 컨텍스트 한계 · 비용↑

생성 — 근거 기반 답

- **무엇**

주입된 근거를 바탕으로 답 생성

- **핵심**

출처를 함께 제시 → 검증 가능

- **효과**

환각↓ · 최신 · 전문 정보 반영↑

- **사례**

OpenEvidence — 동료심사 문헌 (NEJM·JAMA) 기반 의사용 Q&A (약학 RAG)

NOTE 근거에 기반해 답하는 이 방식이 리서치 에이전트의 핵심 패턴이다 .

프롬프트가 동작을 바꾼다

- **프롬프트 = 모델에 주는 입력 전체**

지시 + 맥락 + 예시 + 데이터

- **좋은 프롬프트**

명확한 목표 · 충분한 맥락 · 원하는 형식 명시

- **나쁜 프롬프트**

모호 · 맥락 부족 → 엉뚱하거나 일반적인 답

- **비교 예시**

'코딩 도와줘' vs 'CSV 항목별 합계 코드 만들고 실행 · 확인해줘'

NOTE 같은 모델이라도 프롬프트의 구성이 출력의 품질을 좌우한다.

자주 쓰는 패턴 — 3 가지

- 역할 · 맥락 부여

누가 · 무엇을 · 왜

- 예시 제공 (few-shot)

원하는 입출력 예를 보여주기

- 단계적 사고 (CoT)

단계별로 생각하게

- 사례

GPT-3(2020) — 예시 몇 개 (few-shot) 만으로 번역 · Q&A 수행

NOTE 이어지는 슬라이드에서 세 가지 기법을 하나씩 살펴본다 .

역할 · 맥락 부여

- **방법**

'너는 약학 전문가야' + 상황 · 목적 · 대상 독자

- **효과**

답의 톤 · 깊이 · 용어 수준이 맞춰짐

- **예**

'비전공자에게 설명하듯' vs '전문가용으로'

예시 제공 (few-shot)

- **방법**

원하는 입력→출력 예를 2~3 개 보여준 뒤 본 작업 요청

- **효과**

형식 · 스타일이 안정 (말로 설명하기 어려운 형식에 특히)

- **예**

표 형식 예시 2 개 → 새 데이터도 같은 표로

단계적 사고 (CoT)

- **방법**

' 단계별로 생각해줘 ' / ' 근거를 먼저 정리한 뒤 답해줘 '

- **효과**

복잡한 추론 · 계산의 정확도↑ (생각을 펼치게)

- **주의**

간단한 작업엔 굳이 — 길어지고 느려짐

같은 모델, 다른 출력

- temperature

낮으면 결정적 · 일관, 높으면 다양 · 창의 (불안정)

- 확률적 생성

같은 질문도 매번 다른 답이 나올 수 있다

- 재현성에의 함의

리서치엔 낮은 temperature 가 유리 — 결과 안정

컨텍스트 윈도우

- 한 번에 볼 수 있는 토큰 한계

프롬프트 + 대화 + 자료가 이 한도 안에 들어가야

- 길어지면

비용↑ · 속도↓ · 중간 내용 놓침 (lost in the middle)

- 에이전트와의 관련

외부 메모리 · 검색 · 도구로 한계를 보완

NOTE 현재 Claude 는 최대 100 만 토큰 (책 여러 권 분량) 을 처리하여 긴 문서나 코드를 통째로 분석할 수 있으나, 입력이 길수록 중간 내용을 놓칠 수 있음에 유의해야 한다.

LLM 의 능력과 한계

잘하는 것

- 요약 · 번역 · 분류
- 코드 생성 · 설명
- 텍스트 추출 · 재구성
- 브레인스토밍

주의할 것

- 환각 — 없는 사실 지어냄
- 최신 · 실시간 정보
- 정확한 계산 · 셈
- 일관된 장기 추론

환각 (hallucination) 바로 알기

- **그렇듯하지만 틀린 출력**

출처 · 인용 · 수치를 자신있게 지어내기도

- **왜 생기나**

'그렇듯한 다음 토큰'을 생성할 뿐 사실 검증을 안 함

- **대응**

출처 확인 · 검색 연동 (RAG) · 교차검증 → 리서치 에이전트의 존재 이유

- **실제 사례**

변호사가 ChatGPT 가짜 판례 인용→벌금 (Mata v. Avianca, 2023) · 에어캐나다 챗봇이 없는 환불정책 지어내 배상 (2024)

NOTE 그럴듯함은 사실과 다르므로, 중요한 내용은 반드시 출처로 교차확인해야 한다.

어떤 모델을 쓸까

- 크기 ↔ 성능

큰 모델 = 똑똑하지만 느리고 비쌈

- 작업에 맞게

간단한 작업엔 작고 빠른 모델 , 어려운 작업엔 크고 / 추론 모델

- Claude 예

Opus(최상위) · Sonnet(균형) · Haiku(빠름 · 저비용)

NOTE 참고로 2026 년 6 월 기준 100 만 토큰당 가격은 Opus 가 입력 5 달러 · 출력 25 달러 , Sonnet 이 3·15 달러 , Haiku 가 1·5 달러이며 , 가격은 변동될 수 있다 .

LLM → 에이전트로 가는 길

- LLM 단독의 한계

도구 없음 · 기억 없음 · 한 번의 응답으로 끝

- RAG·검색을 루프로 = 에이전트

찾고 → 검증하고 → 종합 — 리서치 에이전트로

- 다음 : Day 2

에이전트의 정의와 Claude Code 실습

NOTE 오늘 학습한 임베딩과 RAG 가 리서치 에이전트의 토대가 된다 .

핵심 정리 및 다음 과정

핵심 정리

- LLM 의 핵심은 ' 다음 토큰 예측 ' — 확률적 생성이다
- 임베딩 · RAG 로 외부 지식과 결합하면 환각↓ · 최신성↑ (리서치 에이전트의 토대)
- LLM 은 한계가 있다 → 도구 · 검색 · 반복 (에이전트) 이 필요하다

다음 과정 — Day 2

AI 에이전트의 정의 (LLM+tools+loop) 와
리서치 에이전트 개요를 배우고 , Python 과
Claude Code 를 설치 · 설정합니다 .